Agentic AI

1차시: Single Agent

LangChain 기반 AI 에이전트 구축

목차

Part 1: 개념 및 도구

1. Agentic AI란?
2. LangChain & Single Agent 아키텍처

Part 2: 핵심 구성요소

3. Planning & CoT (계획 + 추론)
4. Tool Use (실행)
5. Memory & RAG (기억 + 지식)
6. Self-Reflection (평가)

Part 3: 통합 실습

7. LangGraph 통합 Agent

01

Agentic AI란?

Agentic AI 정의

자율적으로 목표를 달성하기 위해 행동하는 AI 시스템

- 단순 응답이 아닌 **계획-실행-평가** 사이클 수행
- 외부 도구와 상호작용하여 실제 작업 수행
- 환경 변화에 적응하며 동적으로 전략 수정

기존 LLM vs Agentic AI

구분	기존 LLM	Agentic AI
작업 방식	단일 응답 생성	다단계 작업 수행
지식	정적 (학습 데이터)	동적 (실시간 검색)
상호작용	수동적 Q&A	능동적 문제 해결
도구 활용	불가능	API, DB, 코드 실행
자기 개선	없음	Self-Reflection

Agentic AI의 4가지 핵심 능력

Planning	Reflection	Memory	Tools
목표 분해	자가 평가	맥락 유지	외부 연동
실행 순서 결정	오류 수정	과거 학습	기능 확장

02

LangChain & Single Agent 아키텍처

LangChain이란?

LLM 기반 애플리케이션 개발을 위한 **오픈소스 프레임워크**

핵심 특징

- 컴포넌트를 **체인(Chain)** 형태로 연결
- Agent, Memory, Tools 추상화 제공
- 다양한 LLM 벤더 지원

주요 구성요소

- **Models**: LLM 연동
- **Prompts**: 템플릿 관리
- **Chains**: 컴포넌트 연결
- **Agents**: 자율적 의사결정
- **Memory**: 컨텍스트 관리

LangChain 설치 및 기본 설정

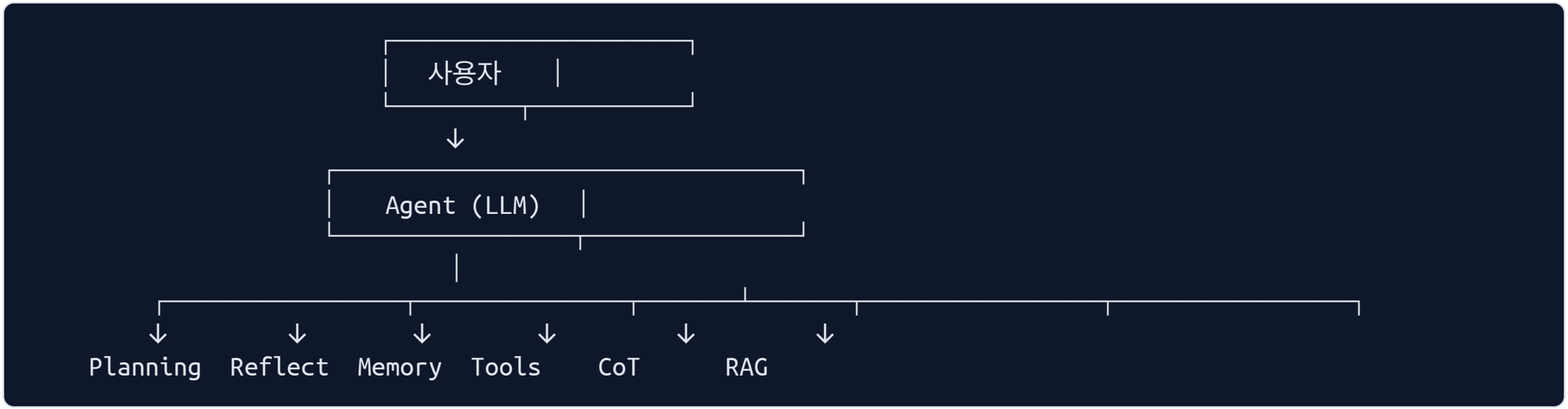
```python
# 설치
pip install langchain langchain-openai langgraph

# 기본 설정
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.7,
    api_key="your-api-key"
)

# 간단한 호출
response = llm.invoke("안녕하세요!")
print(response.content)
```

Single Agent 아키텍처



03

Planning & CoT (계획과 추론)

Planning이란?

복잡한 목표를 **작은 하위 작업으로 분해**하는 능력

계획 수립 방식

방식	설명
Task Decomposition	큰 작업 → 작은 단위로 분해
Subgoal Setting	중간 목표 설정
Ordering	실행 순서 및 의존성 결정

Planning 예시

> *"서울 날씨를 확인하고, 비가 오면 실내 데이트 코스를 추천해줘"*

Agent의 계획 수립

```
1. 서울 현재 날씨 API 호출
2. 날씨 상태 분석 (비 여부 확인)
3. 조건 분기:
    └── 비 O → 실내 장소 검색
    └── 비 X → 야외 장소 검색
4. 장소 정보 정리 및 추천 생성
5. 사용자에게 응답 반환
```

Planning 구현

```
from langchain_core.prompts import ChatPromptTemplate

planning_prompt = ChatPromptTemplate.from_messages([
    ("system", """당신은 작업 계획을 수립하는 AI입니다.
    사용자의 요청을 분석하여 단계별 실행 계획을 세우세요.

    출력 형식:
    1. [단계 1 설명]
    2. [단계 2 설명]
    ..."""),
    ("human", "{task}")
])

planning_chain = planning_prompt | llm
plan = planning_chain.invoke({"task": "최신 AI 논문을 찾아 요약해줘"})
```

Chain of Thought (CoT)

LLM이 **단계별로 추론**하도록 유도하는 프롬프팅 기법

핵심 아이디어

> *"Let's think step by step"*

CoT의 효과

직접 답변	Chain of Thought
질문 → 답변	질문 → 추론1 → 추론2 → ... → 답변
오류 가능성 높음	**추론 과정 검증 가능**

CoT 예시

> *가게에 사과 23개가 있고, 7개를 팔고 15개를 더 받았다면?*

직접 답변

```
답: 31개
(틀릴 수 있음)
```

CoT 답변

```
1. 초기 사과: 23개
2. 7개 판매: 23 - 7 = 16개
3. 15개 추가: 16 + 15 = 31개

답: 31개 ✓
```

CoT 프롬프트 구현

```
cot_prompt = ChatPromptTemplate.from_messages([
    ("system", """당신은 논리적으로 사고하는 AI입니다.
    문제를 해결할 때 다음 단계를 따르세요:

    1. 문제 이해: 주어진 정보 정리
    2. 계획 수립: 해결 방법 구상
    3. 단계별 실행: 각 단계를 명시적으로 수행
    4. 검증: 답이 맞는지 확인
    5. 최종 답변: 결론 제시"""),
    ("human", "{question}")
])

cot_chain = cot_prompt | llm
```

04

Tool Use (도구 사용)

Tool Use란?

Agent가 외부 도구/API를 호출하여 기능을 확장하는 능력

주요 도구 유형

유형	설명	예시
검색	정보 조회	웹 검색, DB 쿼리
계산	연산 수행	수학, 코드 실행
파일	데이터 처리	읽기, 쓰기, 분석
API	외부 연동	REST, GraphQL
실행	코드 수행	Python, SQL

Tool 정의 방법

```python
from langchain.tools import tool

@tool
def search_weather(city: str) -> str:
    """도시의 현재 날씨를 검색합니다."""
    # 실제로는 날씨 API 호출
    return f"{city}의 날씨: 맑음, 20°C"

@tool
def calculate(expression: str) -> str:
    """수학 표현식을 계산합니다."""
    try:
        result = eval(expression)
        return f"결과: {result}"
    except:
        return "계산 오류"

tools = [search_weather, calculate]
```

Agent에 Tool 연결

```python
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

# ReAct 프롬프트 가져오기
prompt = hub.pull("hwchase17/react")

# Agent 생성
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)

# Agent 실행기
agent_executor = AgentExecutor(
    agent=agent, tools=tools, verbose=True
)

# 실행 - 자동으로 적절한 도구 선택 및 호출
result = agent_executor.invoke({
    "input": "서울 날씨 알려주고, 234 * 567 계산해줘"
})
```

05

Memory & RAG (기억과 지식)

Memory가 필요한 이유

LLM은 기본적으로 **상태가 없음** (Stateless)

Memory의 종류

유형	설명	예시
Short-term	현재 대화 컨텍스트	이전 메시지들
Long-term	영구 저장 정보	사용자 선호도
Episodic	특정 이벤트 기억	과거 작업 결과
Semantic	일반 지식	학습된 사실들

Memory 아키텍처

```
┌──────────────────────────────────────────────────────────────────────────┐
│                Agent                    │
│                                         │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │         Working Memory            │  │
│   │       (현재 대화 컨텍스트)          │  │
│   └──────────────────────────┬──────────────────────────────────────┘   │
│                              ↕                 │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │         Long-term Memory          │  │
│   │   ┌──────────────────────┐   ┌──────────────────────┐   │  │
│   │   │  Vector DB  │   │  Key-Value   │   │  │
│   │   │  (의미 검색) │   │   Store      │   │  │
│   │   └──────────────────────┘   └──────────────────────┘   │  │
│   └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
```

LangChain Memory 구현

```python
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

# 대화 버퍼 메모리
memory = ConversationBufferMemory()

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# 대화 진행 - 이전 컨텍스트 유지
conversation.predict(input="내 이름은 철수야")
conversation.predict(input="내 이름이 뭐라고 했지?")
# → "철수라고 하셨습니다"
```

다양한 Memory 전략

```python
# 1. 버퍼 윈도우 메모리 (최근 K개만 유지)
from langchain.memory import ConversationBufferWindowMemory
memory = ConversationBufferWindowMemory(k=5)

# 2. 요약 메모리 (대화를 요약하여 저장)
from langchain.memory import ConversationSummaryMemory
memory = ConversationSummaryMemory(llm=llm)

# 3. 벡터 스토어 메모리 (의미 기반 검색)
from langchain.memory import VectorStoreRetrieverMemory
memory = VectorStoreRetrieverMemory(
    retriever=vectorstore.as_retriever()
)
```

RAG란?

검색(Retrieval) + 생성(Generation) 결합

장점

- 환각(Hallucination) 감소
- 최신 정보 반영 가능
- 도메인 특화 지식 활용

파이프라인

```
질문 → 문서 검색 → 관련 문서 추출 → 컨텍스트 + LLM → 답변
```

RAG 아키텍처

```
┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐
│ Query        │ →  │ Embedding        │ →  │ Vector DB        │
│ (질문)       │    │ Model            │    │ Search           │
└──────────────┘    └──────────────────┘    └──────────────────┘
                                                     │
                                                     ↓
                                          ┌──────────────────┐
                                          │ 관련 문서        │
                                          │ (Top-K)          │
                                          └──────────────────┘
                                                     │
                                                     ↓
┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐
│ 응답         │ ←  │ LLM              │ ←  │ Prompt +         │
│              │    │ Generate         │    │ Context          │
└──────────────┘    └──────────────────┘    └──────────────────┘
```

RAG 구현 - 문서 인덱싱

```python
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# 1. 문서 로드
loader = TextLoader("knowledge_base.txt")
documents = loader.load()

# 2. 청크 분할
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
chunks = splitter.split_documents(documents)

# 3. 벡터 저장소 생성
vectorstore = Chroma.from_documents(chunks, OpenAIEmbeddings())
```

RAG 구현 - 검색 및 생성

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

# 검색기 설정
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

# RAG 프롬프트
rag_prompt = ChatPromptTemplate.from_messages([
    ("system", """다음 컨텍스트를 참고하여 질문에 답하세요.
    컨텍스트에 없는 내용은 "모르겠습니다"라고 답하세요.

    컨텍스트: {context}"""),
    ("human", "{question}")
])

# RAG 체인
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | rag_prompt | llm
)
```

06

Self-Reflection (자기 평가)

Self-Reflection이란?

Agent가 **자신의 출력을 평가**하고 개선하는 능력

핵심 프로세스

```
생성 → 평가 → 수정 → 재평가
          ↑____________|
```

활용 사례

- 코드 생성 후 버그 검토
- 답변의 정확성 검증
- 논리적 일관성 확인
- 품질 기준 충족 여부

Self-Reflection 패턴

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                Agent                      │
│                                           │
│   ┌──────────────────┐      ┌──────────────────────────┐      │
│   │ 초기 응답  │ ──────→ │  자가 평가     │      │
│   │ 생성      │       │  (Critic LLM)  │      │
│   └──────────────────┘      └──────────────────────────┘      │
│                          │              │
│              ┌──────────────────────────┼──────────────────┐   │
│              ↓                          ↓   │
│      [ 만족 → 반환 ]          [ 수정 필요 ]  │
│                              ↓      │
│                         [ 재생성 루프 ]  │
└──────────────────────────────────────────────────────────────────────────────┘
```

Self-Reflection 구현

```
reflection_prompt = ChatPromptTemplate.from_messages([
    ("system", """당신은 AI 응답을 평가하는 비평가입니다.
    다음 기준으로 응답을 평가하세요:
    1. 정확성: 사실에 기반한가?
    2. 완전성: 질문에 충분히 답했는가?
    3. 명확성: 이해하기 쉬운가?

    점수 (1-10)와 개선 제안을 제공하세요."""),
    ("human", "원본 질문: {question}\n응답: {response}")
])

def reflect_and_improve(question, response):
    critique = (reflection_prompt | llm).invoke({
        "question": question, "response": response
    })
    return critique
```

Reflexion 프레임워크

```python
MAX_ITERATIONS = 3

def reflexion_loop(task):
    response = generate_initial_response(task)

    for i in range(MAX_ITERATIONS):
        evaluation = evaluate_response(response)

        if evaluation.score >= 8:  # 품질 기준 충족
            return response

        # 피드백을 반영하여 재생성
        response = improve_response(
            task, response, evaluation.feedback
        )

    return response  # 최대 반복 후 최선의 결과 반환
```

07

LangGraph 통합 실습

LangGraph Agent 상태 정의

```python
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class AgentState(TypedDict):
    task: str
    plan: List[str]
    current_step: int
    results: List[str]
    final_answer: str

def planner(state: AgentState) -> AgentState:
    """작업 계획 수립"""
    plan = planning_chain.invoke({"task": state["task"]})
    return {"plan": parse_plan(plan), "current_step": 0}

def executor(state: AgentState) -> AgentState:
    """현재 단계 실행"""
    step = state["plan"][state["current_step"]]
    result = execute_step(step)
    return {"results": state["results"] + [result]}
```

LangGraph 워크플로우 구성

```python
def reflector(state: AgentState) -> AgentState:
    """결과 검증 및 평가"""
    evaluation = reflection_chain.invoke({
        "task": state["task"], "results": state["results"]
    })
    return {"evaluation": evaluation}

# 그래프 구성
workflow = StateGraph(AgentState)
workflow.add_node("planner", planner)
workflow.add_node("executor", executor)
workflow.add_node("reflector", reflector)

workflow.set_entry_point("planner")
workflow.add_edge("planner", "executor")
workflow.add_edge("executor", "reflector")

agent = workflow.compile()
```

정리: Single Agent 핵심 요소

구성요소	역할	LangChain 구현
Planning	작업 분해 및 계획	Prompt Engineering
Reflection	자가 평가 및 개선	Critique Chain
Memory	컨텍스트 유지	Memory Classes
Tool Use	외부 기능 확장	Tools, Agents
CoT	단계별 추론	Prompt Design
RAG	외부 지식 활용	Retriever + LLM

다음 차시 예고

2차시: Multi-Agent 시스템 (AutoGen)

- AutoGen 프레임워크 소개
- Agent 간 협업 및 대화 패턴
- GroupChat / Supervisor 구조
- 실전 프로젝트: 협업 기반 문제 해결 시스템

참고 자료

공식 문서

- **LangChain**: https://python.langchain.com
- **LangGraph**: https://langchain-ai.github.io/langgraph

주요 논문

- ReAct: Synergizing Reasoning and Acting in Language Models
- Reflexion: Language Agents with Verbal Reinforcement Learning
- Chain-of-Thought: Prompting Elicits Reasoning in LLMs

Q&A

감사합니다

1차시: Single Agent 완료